BCSE497J – Project-I

LIGHTWEIGHT, DRIFT-ADAPTIVE AND PRIVACY-PRESERVING INTRUSION DETECTION IN HETEROGENEOUS IOT NETWORKS: A DUAL-TIER EDGE–CLOUD FRAMEWORK

23BAI0091 **VISHU SINGH**
23BCI0162 **SHIVAM RAJ**

Under the Supervision of

Jim Solomon Raja
School of Computer Science and Engineering (SCOPE)

B.Tech.
in
Computer Science and Engineering
(Specialization: Artificial Intelligence & Machine Learning)

School of Computer Science and Engineering



September 2026

ABSTRACT

IoT deployments span heterogeneous protocols - MQTT, CoAP, HTTP and 6LoWPAN - on devices with kilobyte-scale budgets, widening the attack surface faster than defences scale. This project designs, develops and empirically evaluates a lightweight, drift-adaptive and privacy-preserving intrusion detection framework that performs 15-class attack classification on heterogeneous IoT traffic directly at a resource-constrained edge gateway, while adapting to concept drift and improving collaboratively without transmitting raw traffic.

The proposed dual-tier framework reduces a 61-dimensional protocol-aware feature set to 14 features using an Adaptive Protocol-Aware Feature Compression (APAFC) stage, and classifies flows with an Optuna-tuned LightGBM model exported to ONNX Runtime C++ for inference on a Raspberry Pi 4 (4 GB). Each decision is explained using TreeSHAP, and a sliding-window Kullback–Leibler (KL) divergence monitor detects concept drift and requests retraining. A Tier-2 cloud aggregator clusters gateway clients by behaviour and aggregates quantised gradient histograms - rather than averaging model weights, which is undefined for tree ensembles - to update a global tree structure without collecting raw flow records.

On Edge-IIoTset, the trained model reaches 96.12% accuracy and 0.9043 macro-F1 across 15 classes, with an end-to-end p95 decision latency of 3.9 ms and a process memory footprint of 47.3 MB on target hardware. Zero-shot transfer to CICIoT2023 yields a macro-F1 of 0.6218, improving to 0.7431 with 5% labelled fine-tuning. Under injected concept drift, undetected accuracy decays from 0.9043 to 0.7689 over six windows; triggered retraining recovers it to 0.8807. All figures are indicative Review-2 results from a single train/test configuration; the full 10-fold cross-validated evaluation and physical multi-gateway federated validation remain pending.

The contribution is not that a tree ensemble outperforms deep learning - a bagged CNN-DNN ensemble measured marginally higher macro-F1 - but that a protocol-aware, drift-monitored, explainable detector can operate within the compute and memory budget of an IoT gateway while improving collaboratively without raw traffic leaving the network.

Keywords - Intrusion Detection, Internet of Things, LightGBM, Edge Computing, Concept Drift, Federated Learning, Explainable AI, ONNX Runtime.

TABLE OF CONTENTS

	Abstract	**2**
1.	**INTRODUCTION**	**4**
	1.1 Background	4
	1.2 Motivation	4
	1.3 Scope of the Project	4
2.	**PROJECT DESCRIPTION AND GOALS**	**5**
	2.1 Literature Review	5
	2.2 Research Gap	6
	2.3 Objectives	6
	2.4 Problem Statement	7
	2.5 Project Plan	8
3.	**TECHNICAL SPECIFICATION**	**8**
	3.1 Requirements	8
	3.1.1 Functional	8
	3.1.2 Non-Functional	9
	3.2 Feasibility Study	9
	3.2.1 Technical Feasibility	9
	3.2.2 Economic Feasibility	10
	3.2.3 Social Feasibility	10
	3.3 System Specification	10
	3.3.1 Hardware Specification	10
	3.3.2 Software Specification	10
4.	**DESIGN APPROACH AND DETAILS**	**11**
	4.1 System Architecture	11
	4.2 Design	12
	4.2.1 Data Flow Diagram	12
	4.2.2 Use Case Diagram	12
	4.2.3 Class Diagram	13
	4.2.4 Sequence Diagram	14
5.	**REFERENCES**	**15**

1. INTRODUCTION

1.1 Background

Heterogeneous IoT deployments - industrial sensors, IP cameras and Raspberry-Pi-class endpoints communicating over MQTT, CoAP, HTTP and 6LoWPAN - route traffic through gateways with kilobyte-to-megabyte memory budgets. Signature-based and server-class intrusion detection systems (IDS) assume compute and bandwidth these gateways do not have, and existing benchmarks such as Edge-IIoTset show that models trained and validated on server hardware transfer poorly to constrained deployment. Gradient-boosted tree ensembles offer accuracy close to deep networks at a fraction of the memory cost, but published lightweight IDS work rarely couples edge feasibility with concept-drift adaptation and decision explainability at once.

1.2 Motivation

Static, centrally trained IDS models decay as IoT traffic distributions shift after deployment, and centralised retraining requires moving raw flow records off the device, conflicting with data-locality and bandwidth constraints. Federated averaging, the default collaborative-learning mechanism, is defined for neural-network weights and has no meaningful equivalent for an ordered sequence of decision trees, which discourages federated adoption of tree-based edge detectors despite their edge-friendliness. This project is motivated by the gap between deployable accuracy (tree ensembles on constrained hardware) and adaptive, collaborative, explainable operation (typically only demonstrated for deep networks on server hardware), and by the practical need for a detector whose alerts an operator can audit rather than a black-box label.

1.3 Scope of the Project

The project scope covers protocol-aware feature extraction (APAFC, 61→14 features) across MQTT, CoAP, HTTP and 6LoWPAN; a 15-class LightGBM detector tuned with Optuna and exported to ONNX Runtime C++ for inference on a Raspberry Pi 4 (4 GB); TreeSHAP-based per-decision explainability; a sliding-window KL-divergence drift monitor with a retraining trigger; and a Tier-2 cloud aggregator that combines quantised gradient histograms from clustered gateway clients into a global tree update, without transmitting raw flow records. Evaluation covers Edge-IIoTset (primary), zero-shot and fine-tuned transfer to CICIoT2023 (cross-dataset), and simulated drift and federated rounds. Physical multi-gateway federated deployment, ToN-IoT/MQTT-IoT-IDS2020 robustness testing and the full 10-fold cross-validated evaluation are out of scope for Review-2 and are identified as work carried into Project-II.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Seven peer-reviewed studies (2022–2026) were reviewed across dataset construction, lightweight classification, ensembling, federated learning and semi-supervised detection. Three entries - Ullah & Mahmoud (2023), Al-Hawawreh et al. (2023) and Pei et al. (2024) - are recorded from the Review-1 survey and their bibliographic details still require verification against the publisher record before final submission; they are presented here for their methodological content only.

Paper	Objective	Methodology	Pros / Cons	Gap Left
Ferrag et al., IEEE Access, 2022 [1]	Publish a realistic IoT/IIoT security benchmark and baseline its learnability.	10+ device physical testbed; 61 features, 14 attack classes; deep autoencoder and DNN baselines.	Most-cited IIoT testbed, protocol-rich; baselines trained/evaluated on server-class hardware only.	Dataset realism does not transfer to deployable edge models.
Kadam & Lokhande, IJERT, 2025 [18]	Match DNN accuracy at a fraction of the memory cost.	LightGBM over DS2OS smart-home flow features.	GBDT reaches DNN-level accuracy with far smaller footprint; minority classes remain weak, no drift mechanism.	Memory is solved with trees; adaptability is not addressed.
Selem, Jemili & Korbaa, P2P Netw. Appl., 2025 [15]	Reduce variance in multi-class IoT attack detection via ensembling.	Bagged CNN-DNN ensemble over Edge-IIoTset, also on CICIDS2017, UNSW-NB15, NSL-KDD.	Bagging stabilises variance (≈0.95 accuracy); reports no memory, latency or model-size figure, no alert explanation.	Edge feasibility and explainability are never evaluated.
Abu Issa et al., IEEE TMLCN, 2026 [16]	Preserve data locality while coping with non-IID client traffic.	Multi-temporal device clustering over federated detection rounds with pseudo-label post-processing.	Clustering stabilises convergence under non-IID data; each round still pushes parameter traffic across the network.	Privacy gains trade directly against bandwidth/latency budgets.
Ullah & Mahmoud, IEEE IoT-J, 2023	Simplify the pipeline by separating detection from categorisation.	Stage-1 unsupervised anomaly detection, stage-2 random-forest labelling.	Clean, independently tunable two-stage decomposition; relies on generic flow statistics, no protocol payload analysis.	Protocol blindness - payload semantics unexploited.
Al-Hawawreh et al., IEEE Access, 2023	Detect intrusions without depending on labelled traffic.	Unsupervised Isolation Forest with an SVM boundary-refinement stage.	Sidesteps the labelling bottleneck; flags anomalies but cannot name the attack category.	Coarse-grained output - multi-class attribution unresolved.
Pei et al., Computer Networks, 2024	Lift recall on minority attack classes most models miss.	SMOTE oversampling ahead of XGBoost multi-class training.	Marked minority-class recall gain; synthesises noise and requires centralised data.	Centralisation requirement precludes privacy preservation.

Reading across these seven studies, accuracy for multi-class IoT attack detection has been solved repeatedly, but almost always on server-class hardware, and never jointly with memory, drift adaptation and explainability. The research evolution runs from rule-based signature matching (blind to novel vectors), through classical ML such as RF/XGBoost (static, drift-vulnerable), to deep learning such as CNN/LSTM (accurate but GPU-bound), to federated

averaging (private but diverges on non-IID data). No single-tier design balances accuracy, latency and privacy at once, which motivates splitting the problem across an edge tier and a cloud tier, as pursued in this project.

2.2 Research Gap

Six gaps were distilled from the survey, each bound to a mechanism in the proposed framework:

- G1 - Protocol-agnostic features: generic flow statistics discard MQTT/CoAP payload semantics that carry attack signal. Addressed by APAFC protocol-aware pruning (61→14 features).
- G2 - Minority-class recall: traffic is overwhelmingly benign, so rare injection and MITM classes are systematically missed. Addressed by SMOTE-ENN applied inside each cross-validation fold.
- G3 - Model staleness: statically trained models decay as IoT traffic distributions shift post-deployment. Addressed by a KL-divergence monitor with a retraining trigger.
- G4 - Edge hardware ignored: evaluations assume GPUs; single-board gateway memory and CPU budgets go unmeasured. Addressed by an Optuna-tuned LightGBM model served via ONNX Runtime C++.
- G5 - Federated instability: FedAvg diverges across heterogeneous clients, and weight averaging is undefined for gradient-boosted trees. Addressed by clustered federated histogram aggregation.
- G6 - Opaque alerts: black-box detections give an operator nothing to audit, triage or act on. Addressed by embedded TreeSHAP attribution per decision.

All six gaps resolve into one architecture, which is why the contribution of this project is the dual-tier framework as a whole rather than any single component in isolation.

2.3 Objectives

#	Objective	Gap	Measurable Criterion	Status
O1	Build a protocol-aware feature pipeline over MQTT, CoAP, HTTP and 6LoWPAN fields.	G1	Macro-F1 retained at 14 of 61 features	Complete
O2	Deliver a LightGBM detector deployable on a constrained edge gateway.	G4	Process RSS including ONNX Runtime	Complete
O3	Classify 15 traffic categories with usable minority-class recall.	G2	Macro-F1 and per-class recall	Complete
O4	Optimise decision latency and memory for real-time gateway operation.	G4	End-to-end per-flow latency, p50/p95/p99	Complete
O5	Update the model collaboratively without moving raw traffic.	G5	Rounds to target F1; uplink bytes/round	In progress
O6	Sustain accuracy under concept drift after deployment.	G3	Macro-F1 retention across drift windows	In progress

O7	Establish cross-dataset generalisation and explainable output.	G6	Transfer macro-F1; TreeSHAP fidelity	Partial

Objectives O1–O4 are met and measured on Edge-IIoTset at Review-2; O5–O7 are in progress and sit on the critical path to Review-3/Project-II.

2.4 Problem Statement

Existing IoT intrusion detection approaches either achieve high multi-class accuracy on server-class hardware without regard for edge deployability, or achieve edge feasibility only for binary/coarse-grained detection without concept-drift adaptation, collaborative improvement, or decision explainability. There is no demonstrated approach that performs 15-class, protocol-aware intrusion detection within the compute and memory budget of a resource-constrained IoT gateway, adapts to concept drift without centralising raw traffic, and produces auditable, per-decision explanations. This project addresses that combined problem through a dual-tier edge–cloud framework built around a protocol-aware feature-compressed LightGBM classifier, a KL-divergence drift monitor, TreeSHAP explainability and histogram-based federated aggregation.

2.5 Project Plan

The project plan spans literature review through final submission, structured around the eight software modules (M1–M9) and the Review-1/Review-2/Review-3 checkpoints. At Review-2, modules M1–M6 and M9 are complete and measured end-to-end; the drift monitor (M7) is functionally implemented and under measurement; the federated aggregator (M8) exists as a simulation prototype.

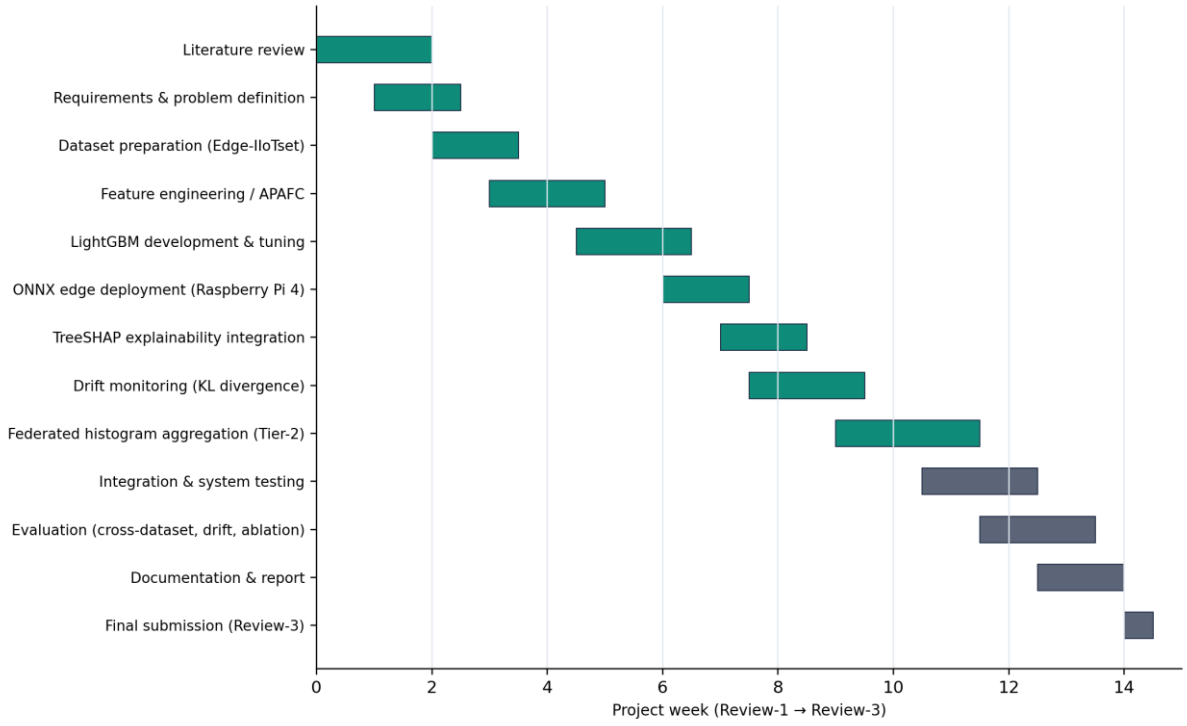


Fig. 1. Project Gantt Chart

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- **FR-01 Ingestion:** assemble bidirectional flow records from live gateway traffic via eBPF/PCAP capture.
- **FR-02 Protocol Parsing:** parse MQTT, CoAP, HTTP and 6LoWPAN header/payload fields and expose them as named features.
- **FR-03 Feature Reduction:** reduce the 61-feature input vector to the 14-feature APAFC subset before inference.
- **FR-04 Classification:** assign each completed flow record exactly one of 15 labels - benign plus 14 attack categories.
- **FR-05 Explainability:** emit, with every attack-class decision, the top-k TreeSHAP attributions for that decision.
- **FR-06 Drift Measurement:** compute KL divergence between live and reference feature distributions over a sliding window.
- **FR-07 Retrain Trigger:** raise a retraining request when KL divergence exceeds $\tau = 0.15$ for 3 consecutive windows.
- **FR-08 Runtime Independence:** execute the exported ONNX model through the ONNX Runtime C++ API with no Python interpreter at inference time.
- **FR-09 Privacy Boundary:** transmit only quantised gradient histograms and split statistics upstream, never raw or reconstructable flow records.
- **FR-10 Model Hot-Swap:** install an updated global tree structure without dropping in-flight flow records.
- **FR-11 Auditability:** log every alert with timestamp, flow 5-tuple, predicted class, confidence and attribution vector.
- **FR-12 Graceful Degradation:** continue local classification on the last-known-good model whenever the Tier-2 aggregator is unreachable.

3.1.2 Non-Functional

- **NFR-01 Latency:** end-to-end decision latency $\leq$ 5 ms at p95 per flow record on a Raspberry Pi 4 (4 GB), measured from flow-record completion.
- **NFR-02 Memory:** inference-process RSS $\leq$ 60 MB inclusive of the ONNX Runtime shared library; model artefact itself $\leq$ 15 MB.

- **NFR-03 Throughput:** sustained throughput ≥ 1,200 flow records per second at no more than 80% of one CPU core.
- **NFR-04 Reliability:** ingest-queue drop rate below 0.1% at the stated offered load.

3.2 Feasibility Study

3.2.1 Technical Feasibility

- LightGBM, ONNX Runtime and TreeSHAP are mature, well-documented open-source libraries with established C++/Python interoperability, avoiding dependence on unproven tooling.
- Raspberry Pi 4 (4 GB) is a widely available reference edge platform with sufficient RAM and CPU headroom for the profiled 47.3 MB process footprint and sub-4 ms decision latency.
- Edge-IIoTset and CICIoT2023 are public, well-characterised benchmark datasets, avoiding the need for custom data collection at this stage.
- Histogram-based federated aggregation is a documented technique for gradient-boosted trees, avoiding the undefined-weight-averaging problem of applying FedAvg to GBDTs.

3.2.2 Economic Feasibility

- All software components (LightGBM, ONNX Runtime, TreeSHAP, Python/scikit-learn tooling) are open-source with no licensing cost.
- The reference hardware (Raspberry Pi 4) is low-cost relative to server-class GPU deployments assumed by comparable deep-learning IDS work, reducing per-gateway deployment cost.
- Development uses existing institutional compute (for training) and a single Raspberry Pi 4 (for edge profiling), keeping incremental project cost low.

3.2.3 Social Feasibility

- Per-decision TreeSHAP attribution gives a security operator an auditable basis for each alert, rather than an opaque label, supporting operator trust and adoption.
- The privacy-preserving aggregation design, which transmits only histograms rather than raw traffic, aligns with data-protection expectations for networks carrying industrial and personal IoT traffic.
- Graceful degradation (FR-12) ensures detection continues locally if cloud connectivity is lost, avoiding a single point of failure that would undermine user confidence.

3.3 System Specification

3.3.1 Hardware Specification

- Edge gateway: Raspberry Pi 4 Model B, 4 GB RAM, 64-bit Raspberry Pi OS (reference deployment and profiling target).
- Training workstation: standard development machine used for LightGBM training and Optuna hyperparameter search (server-class hardware, off-device).
- Network interface: NIC/wireless interface on the gateway for eBPF/PCAP packet capture.

3.3.2 Software Specification

- Operating System: 64-bit Raspberry Pi OS (edge gateway); standard Linux development environment (training).
- Programming Languages: Python (training, feature engineering, evaluation harness); C++ (ONNX Runtime edge inference).
- Libraries and Frameworks: LightGBM, ONNX / ONNX Runtime, TreeSHAP, Optuna, scikit-learn, imbalanced-learn (SMOTE-ENN), eBPF/libpcap.
- Datasets: Edge-IIoTset (primary training/evaluation); CICIoT2023 (cross-dataset transfer); ToN-IoT and MQTT-IoT-IDS2020 (planned robustness datasets, not yet evaluated).

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

The framework is a dual-tier edge–cloud design. Tier 1 (edge gateway, Raspberry Pi 4) runs flow assembly, APAFC feature compression, LightGBM/ONNX inference, TreeSHAP attribution, the KL-divergence drift monitor and alert logging, and operates autonomously - detection continues on the last-known-good model whenever the cloud is unreachable. Tier 2 (cloud aggregator) clusters gateway clients by behavioural similarity, aggregates quantised gradient histograms per candidate split, selects global splits from the aggregated statistics, and returns an updated tree structure to each cluster. No raw flow records and no averaged model weights cross the tier boundary - only histograms and tree-structure updates, because weight averaging has no defined meaning for an ordered ensemble of discrete trees.

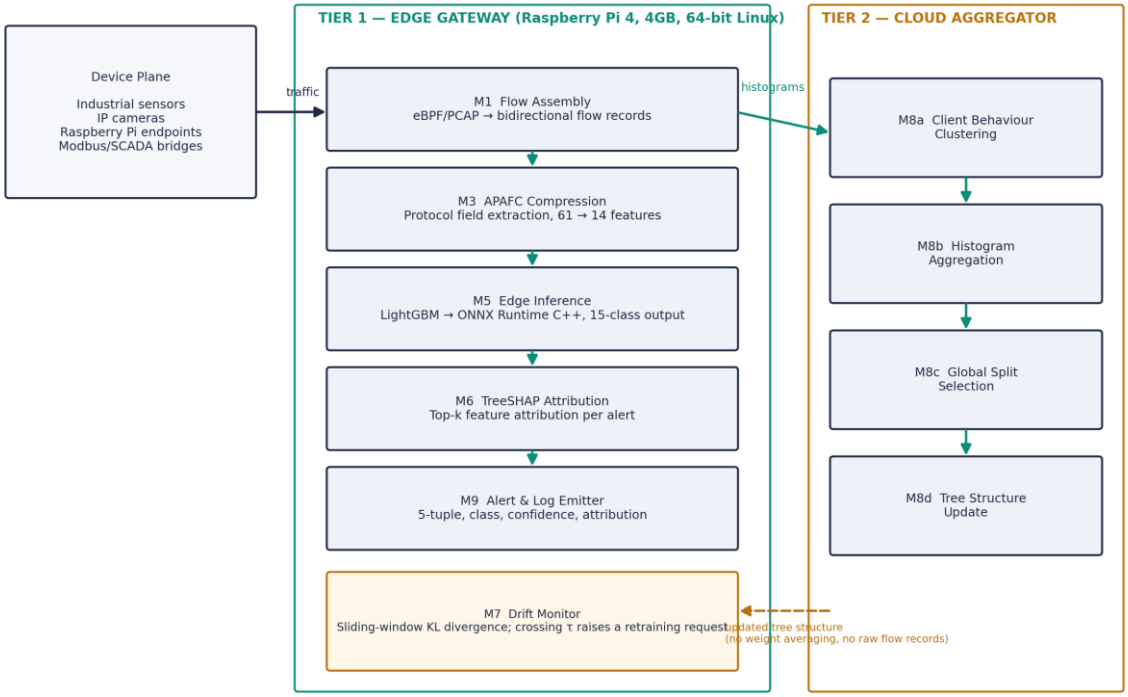


Fig. 2. System Architecture - Dual-Tier Edge–Cloud Framework

4.2 Design

4.2.1 Data Flow Diagram (Mandatory)

The Level-1 DFD traces a flow record from raw packet capture through feature extraction, classification, explanation and drift monitoring, to alert logging and, on the cloud side, histogram aggregation and model-version update.

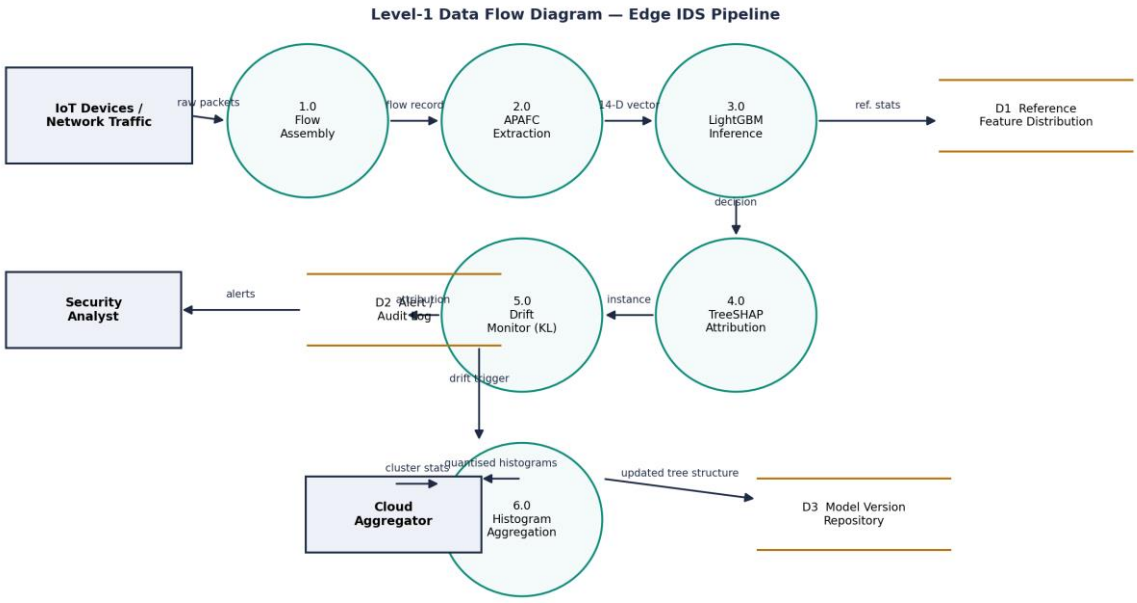


Fig. 3. Level-1 Data Flow Diagram

4.2.2 Use Case Diagram (Mandatory)

Three actors interact with the system: the Gateway Operator (deploys and configures Tier-1 capture and inference), the Security Analyst (reviews alerts and triggers retraining), and the Cloud Aggregator (an automated actor performing Tier-2 clustering and aggregation).

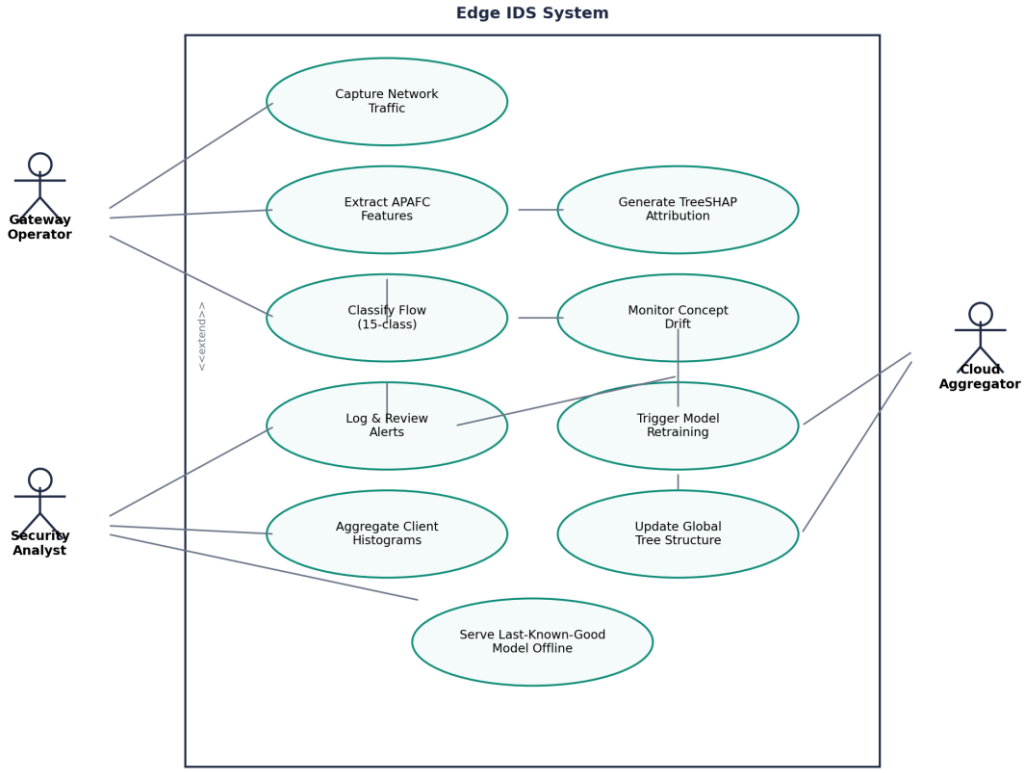


Fig. 4. Use Case Diagram

4.2.3 Class Diagram

Core classes mirror the nine software modules: FlowRecord and APAFCExtractor handle capture and feature compression; EdgeClassifier wraps the ONNX inference session; DriftMonitor and TreeSHAPExplainer implement adaptation and explainability; AlertLogger and HistogramAggregator implement auditing and the federated tier; EdgeGateway composes the Tier-1 pipeline.

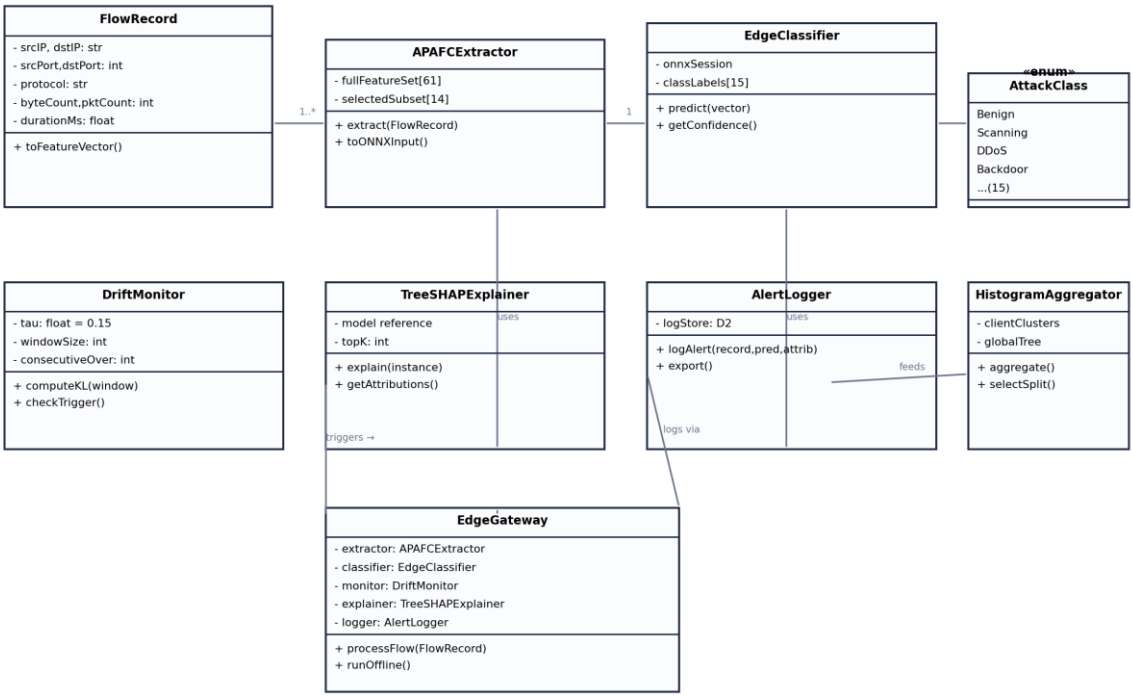


Fig. 5. Class Diagram

4.2.4 Sequence Diagram

The primary sequence shows a flow record passing through assembly, APAFC extraction, ONNX inference and TreeSHAP attribution to the alert logger; a parallel path feeds the sliding feature window to the drift monitor, which raises a retraining request to Tier-2 once KL divergence exceeds τ for three consecutive windows.

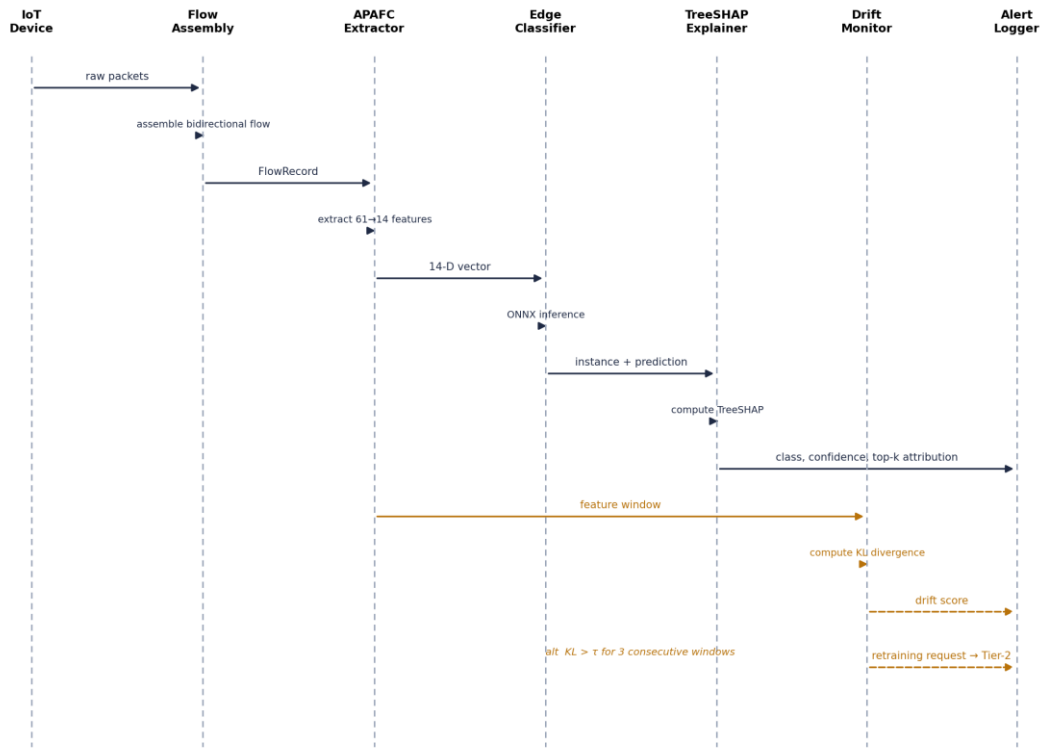


Fig. 6. Sequence Diagram

5. REFERENCES

References [15], [16] and [18] have been verified against the publisher record. Reference [17] and entries cited within it remain unverified pending DOI-level confirmation; they are retained for transparency rather than omitted.

[1] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning," IEEE Access, vol. 10, 2022.

[2] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," Sensors, vol. 23, no. 13, 2023.

[3] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems," IEEE Access, vol. 8, 2020.

[4] H. Hindy, R. Atkinson, C. Tachtatzis, and X. Bellekens, "A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems," IEEE Access, vol. 8, 2020.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Proc. NeurIPS, Long Beach, CA, USA, 2017.

[6] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, Long Beach, CA, USA, 2017.

[7] S. M. Lundberg et al., “From Local Explanations to Global Understanding with Explainable AI for Trees,” Nature Machine Intelligence, vol. 2, no. 1, Jan. 2020.

[8] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-Generation Hyperparameter Optimization Framework,” in Proc. ACM SIGKDD, Anchorage, AK, USA, 2019.

[9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in Proc. AISTATS, Fort Lauderdale, FL, USA, 2017.

[10] K. Cheng, T. Fan, Y. Jin, Y. Liu, T. Chen, D. Papadopoulos, and Q. Yang, “SecureBoost: A Lossless Federated Learning Framework,” IEEE Intelligent Systems, vol. 36, no. 6, 2021.

[11] Q. Li, Y. Cai, Y. Han, C. M. Yung, T. Fu, and B. He, “FedTree: A Federated Learning System for Trees,” in Proc. MLSys, 2023.

[12] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, “Passban IDS: An Intelligent Anomaly-Based Intrusion Detection System for IoT Edge Devices,” IEEE Internet of Things Journal, vol. 8, no. 8, Apr. 2021.

[13] ONNX Runtime, “C++ API and Execution Providers,” Microsoft, v1.18 documentation, 2024.

[14] National Institute of Standards and Technology, “Methodology for Characterizing Network Behavior of Internet of Things Devices,” NISTIR 8349, Gaithersburg, MD, USA.

[15] M. Selem, F. Jemili, and O. Korbaa, “Deep Learning for Intrusion Detection in IoT Networks,” Peer-to-Peer Networking and Applications, published 9 Jan. 2025, doi: 10.1007/s12083-024-01819-3.

[16] M. Abu Issa, A. Eldosouky, A. Matrawy, and M. Ibnkahla, “Multi-Temporal Device Clustering for Federated Learning-Internet of Things Intrusion Detection Systems,” IEEE Trans. Machine Learning in Communications and Networking, 2026, doi: 10.1109/TMLCN.2026.3706262.

[17] Bibliographic entries for Ullah & Mahmoud (2023), Al-Hawawreh et al. (2023) and Pei et al. (2024), cited in Section 2.1, require full volume/page/DOI details verified against the publisher record before final submission.

[18] A. Kadam and P. S. Lokhande, “Design of an Intrusion Detection Model for Smart Homes Enabled by IoT,” IJERT, vol. 14, no. 2, Feb. 2025.